

Feature Set — RISC-V-Based Spectrum Analyzer SoC

This table focuses on the main features of your project, what each feature does, and which IPs are used to implement it and why. We can refine the features before converting the table into a copy-paste format.

S.No.	Feature	Feature Description	IPs Used	Why These IPs Are Used
1	Analog Signal Acquisition	Acquires an external analog signal and obtains digital samples for further processing.	External ADC, ADC Interface	The external ADC converts the analog signal into digital samples. The ADC Interface receives and transfers those samples into the SoC.
2	Sample Buffering	Temporarily stores acquired samples so they can be supplied to the processing blocks in the required order.	ADC Interface, FFT Memory Interface	The ADC Interface handles incoming samples and any implemented FIFO buffering. The FFT Memory Interface provides access to memory used for FFT data.
3	Digital Noise Filtering	Filters unwanted noise or frequency components from the acquired signal before spectral analysis.	FIR Filter	The FIR accelerator processes input samples using filter coefficients to produce filtered samples.

4	Frequency Spectrum Analysis	Transforms time-domain samples into frequency-domain data, enabling the frequency components of a signal to be analyzed.	FFT Accelerator	The FFT performs the Fast Fourier Transform on the supplied samples.
5	FFT Data Storage and Access	Stores or accesses the data used by FFT processing, including input or output data as supported by the design.	FFT Memory Interface	Provides the memory-access path for FFT data. The exact input/output storage arrangement depends on the implemented RTL.
6	FFT Processing Control	Starts FFT processing and provides status information about processing progress and completion.	FFT Control	The control interface provides the start command and status registers used to coordinate FFT execution.
7	Software-Based System Control	Allows the processor to configure and coordinate the connected peripherals and DSP accelerators.	RISC-V CPU, AXI Interconnect, peripheral control interfaces	The CPU executes software, while the interconnect routes transactions to the selected IP.

8	Host Communication	Enables serial communication between the SoC and a host PC for supported commands, status information and result transfer.	UART	The UART provides the serial transmit and receive interface. Software must handle the actual data exchange and result interpretation.
9	Digital Input/Output and Status Indication	Provides digital input and output signals for external control or status indication.	GPIO	GPIO allows software to read input pins and control output pins.
10	Timer-Based Timing Support	Provides counting and timing functions for software or other supported system operations.	Timer	The timer provides the counting, comparison or interrupt functions implemented in the selected RTL.
11	Integrated Signal-Processing Workflow	Coordinates sample acquisition, filtering, FFT execution, memory access and communication of results to the host.	RISC-V CPU, AXI Interconnect, ADC Interface, FIR, FFT Control, FFT Memory Interface, UART	These blocks work together to form the complete spectrum-analysis processing and control flow.

A few points to settle before finalizing

1. The core features are signal acquisition, sample buffering, digital filtering, FFT-based frequency analysis, and result access. The remaining rows describe the supporting control and communication capabilities.

2. The ADC itself may be external. Unless your RTL includes an ADC converter, describe the ADC Interface as receiving digital samples rather than performing analog-to-digital conversion.
3. Spectrum display is not automatically an SoC feature. If a PC application plots the FFT results, we can list it as a host-side software feature separately.
4. GPIO and Timer are supporting peripherals, not the primary spectrum-analysis functions.
5. We should distinguish between features that are already implemented and integrated, and those that are planned but not yet verified.

1. Data Path of the Spectrum Analyzer SoC

The data path explains how the actual signal samples travel through your SoC, how they are processed by the hardware IPs, where the results are stored, and how they reach the host PC.

Your project is titled “Design and RTL Implementation of a RISC-V Based Spectrum Analyzer SoC with Hardware DSP Acceleration.” Its main purpose is to take a digitized signal, process it using dedicated DSP hardware, and make its frequency-domain information accessible to software.

Overall data path

1. External Analog Signal

The real-world signal to be analyzed

2. External ADC

Converts the analog signal into digital samples

3. ADC Interface IP

Receives and manages the digital sample data

4. FIR Filter IP — optional/configuration-dependent

Filters unwanted frequency components or noise before FFT processing

5. FFT Accelerator

Transforms time-domain samples into frequency-domain results

6. FFT Memory Interface / Data Memory

Provides the memory access path for FFT data and results, according to the implemented memory protocol

7. RISC-V CPU

Reads and interprets the stored results through the system's supported bus path

8. UART → Host PC

Transfers results or status information to host-side software for display and analysis

Conceptual end-to-end flow. The exact sample handshakes, memory routing, and CPU-to-peripheral bus path must follow your actual RTL integration. The external ADC is separate from the ADC Interface IP unless your implementation explicitly includes conversion hardware.

Detailed explanation of each data-path stage

1. External analog signal and ADC

The input may be a sensor signal, audio waveform, vibration signal, or another electrical signal whose frequency content needs to be studied.

The external Analog-to-Digital Converter (ADC) samples the analog waveform and converts it into numerical values. These digital samples represent the signal's amplitude at successive points in time.

For example, an input waveform might be represented by a sequence of sample values:

`x[0], x[1], x[2], x[3], ...`

The sampling rate is important because it determines which input frequencies can be represented and analyzed.

2. ADC Interface IP

The ADC Interface receives digital samples from the external ADC and provides the interface through which the rest of the system can access them.

Depending on the RTL implementation, it may handle sample-valid signals, sample buffering, status reporting, or interrupt generation. It does not automatically perform analog-to-digital conversion itself.

Its role: bridge the incoming sample source and the SoC's digital processing system.

3. FIR Filter IP

The Finite Impulse Response (FIR) filter processes the incoming sample sequence using programmed filter coefficients.

It can be configured to attenuate unwanted frequency components, such as out-of-band noise, before the signal enters the FFT stage. Whether filtering is performed for every acquisition depends on how the FIR IP is configured and controlled.

Its role: condition the digital signal before frequency analysis.

4. FFT Accelerator

The Fast Fourier Transform (FFT) accelerator is one of the central processing blocks in your project.

The input samples represent the signal in the time domain. The FFT converts a block of these samples into a frequency-domain representation.

The output contains complex frequency-bin values, conventionally represented by real and imaginary components. Their magnitudes can be calculated to understand the relative strength of different frequency components.

For example, a signal containing two prominent sinusoidal components may produce two prominent peaks in its frequency spectrum, subject to sampling, windowing, resolution, and scaling.

Its role: perform computationally intensive frequency analysis in dedicated hardware rather than requiring the processor to calculate the entire FFT in software.

5. FFT Memory Interface and data storage

The FFT Memory Interface provides the access path to the memory used by the FFT subsystem. The exact arrangement of input samples, intermediate data, and output results depends on the implemented memory organization.

The memory interface and the FFT Control interface serve different purposes:

- FFT Memory Interface: supports access to FFT data and the associated memory region.
- FFT Control Interface: allows software to initiate processing and inspect its status.

They should be documented as separate functional interfaces even though both belong to the FFT subsystem.

6. RISC-V CPU and result access

The RISC-V processor coordinates the system by executing software that configures the peripherals, initiates operations, checks status, and accesses the processed data.

In the intended architecture, the CPU reaches memory and peripherals through the supported system bus and interconnect paths. It does not need to perform the FFT arithmetic itself.

Once results are available, the CPU can retrieve the required data and prepare it for communication.

7. UART and host PC

UART provides serial communication between the SoC and a host computer.

The CPU can send selected frequency-bin values, calculated magnitudes, or processing status to the PC. Host-side software can then format or plot the data as a spectrum.

An important distinction: the SoC performs the signal-processing work, while the actual graph or spectrum display is a host-side software function unless you implement a display subsystem in hardware.

2. Control Path of the Spectrum Analyzer SoC

The control path explains how the CPU configures the IPs, tells them when to operate, checks whether processing has completed, and coordinates the overall workflow.

The data path carries the signal samples and results. The control path manages when and how the different blocks perform their tasks.

Overall control path

RISC-V CPU

Executes firmware and issues control transactions

System Bus / Interconnect

Routes transactions to the addressed memory or peripheral

Address Decoder and IP Interfaces

Select the intended peripheral and access its control/status interface

ADC Interface

Sample acquisition configuration and status

FIR Filter

Enable, coefficients, and status

FFT Control

Start command and busy/done status

Timer / GPIO / UART

Timing, digital I/O, and host communication

This is a conceptual control-path view. Your actual SoC may use different bus segments for CPU control transactions and FFT memory accesses. The exact protocol and connections must match the integrated RTL.

Detailed control sequence

Step 1: CPU initialization

When the system starts, the RISC-V CPU executes its firmware. The firmware initializes the required peripherals and configures the signal-processing pipeline.

This may include setting FIR coefficients, enabling the required interfaces, configuring UART communication, and preparing the FFT subsystem.

Step 2: Configure and acquire samples

The CPU configures the ADC Interface and the external sample source as required by their interfaces. Digital samples then enter the acquisition path.

The CPU can check relevant status information to determine whether samples are available or whether the required sample block has been collected. The exact mechanism—polling, interrupts, or another handshake—depends on the implemented IP.

Step 3: Configure the FIR filter

If filtering is enabled, the CPU programs the FIR filter's coefficients and control settings. The FIR IP then processes samples according to its RTL-defined interface and operating mode.

The CPU's role is to configure and coordinate the operation; the filter's hardware performs the filtering arithmetic.

Step 4: Start the FFT accelerator

The CPU prepares the FFT input data and issues the appropriate start command through the FFT Control interface.

In the previously discussed FFT control-register design, the control register uses a start bit and the status register reports busy and done states. These details should be checked against the exact RTL revision used in the final build.

Step 5: Monitor FFT execution

After starting the FFT, the CPU checks the status interface or uses the implemented completion mechanism to determine when processing finishes.

The CPU should not treat a start command as proof that the FFT results are already available. It must respect the actual busy/completion behavior.

Step 6: Access the results

Once processing is complete, the CPU accesses the results through the implemented FFT memory/data path. It can select the values needed for transmission, such as frequency-bin values or real and imaginary components.

The FFT accelerator performs the transform; the CPU manages access to the resulting data.

Step 7: Transmit the results through UART

The CPU sends the selected results or status information through UART to the host PC. Host-side software can decode the received data and, if implemented, calculate magnitudes and plot the spectrum.

Step 8: Use supporting peripherals

- Timer: can provide timing or periodic events when configured for that purpose.
- GPIO: can provide digital inputs/outputs, status indications, or control signals.
- UART: carries information between the SoC and the host PC.

These peripherals support the main signal-processing workflow but do not replace the ADC, FIR, or FFT functions.

Data path vs. control path

Aspect	Data path	Control path
Main purpose	Moves and processes signal samples and results	Configures, starts, and monitors IP operation
Main components	ADC, ADC Interface, FIR, FFT, FFT memory	RISC-V CPU, bus/interconnect, control/status interfaces
Information carried	Sample values, filtered samples, FFT results	Configuration writes, start commands, status reads
Hardware acceleration	Performs filtering and FFT arithmetic	Coordinates the hardware operations
Final role	Produces frequency-domain data	Makes the processing sequence manageable by software

The two paths work together: the control path initiates and coordinates processing, while the data path performs the actual signal transformation and transports its results.

3. Overall project description: What, why, problem, and applications

What does your project do?

Your project develops a RISC-V-based System-on-Chip (SoC) that combines a programmable processor with dedicated hardware for digital signal processing.

It is designed to analyze the frequency content of an incoming signal. The signal is digitized by an external ADC, passed through the ADC Interface, optionally filtered using the FIR IP, and processed by the FFT accelerator. The resulting frequency-domain data is stored or accessed through the FFT memory path. The RISC-V CPU coordinates the process and communicates selected results to a host PC using UART.

The key idea is to combine software programmability with hardware acceleration in one integrated system.

What problem are you solving?

Many real-world signals contain multiple frequency components. Looking at a waveform in the time domain may not make it easy to identify which frequencies are present or which components are dominant.

A spectrum analyzer addresses this problem by transforming sampled signal data into the frequency domain.

Your project focuses on implementing this processing capability as an integrated SoC, rather than treating the ADC interface, filtering, FFT computation, processor control, memory access, and communication as entirely separate demonstrations.

The project addresses three related engineering challenges:

1. Frequency analysis: converting time-domain samples into frequency-domain information using an FFT accelerator.
2. Processing efficiency: using dedicated hardware for computationally intensive DSP operations instead of relying entirely on software execution.
3. System integration: connecting the processor, DSP IPs, memory interface, interconnect, and supporting peripherals so that software can control the complete workflow.

The project does not automatically identify every type of signal or diagnose a fault. It provides the underlying frequency-analysis capability that can support such applications when suitable software and interpretation methods are added.

Why is this useful in the real world?

Frequency analysis is useful because different physical systems produce signals with characteristic frequency components. Measuring those components can help engineers understand system behavior, identify interference, and monitor changes over time.



1. Communication systems and RF analysis

Frequency analysis can help inspect signals, identify spectral components, and investigate interference. Your SoC demonstrates the underlying FFT-based analysis; practical RF measurements also require appropriate analog front-end hardware and sampling specifications.

! [Sensores industriales para monitoreo de vibraciones en equipos rotativos: tecnologías clave para mantenimiento predictivo

- Revista Tecnología Minera] (https://images.openai.com/static-rsc-4/zkpRwM-sUy_Jw6s1E-7GfdDVCnW22ExITBVuhgtYXPw4x5ohTp934Slyg-C68PE1w3W-Tf-yGL3an4v03NESvA1ZbTd7HCSJ4Q5Twv4I7sdACURWjbBDPC6HuulUCbExTEFFfcPqOfOAmHOSIDjDmvs6mWES31XgMezwSK06Gvc?purpose=inline)

2. Industrial machinery monitoring

Vibration signals can contain frequency patterns associated with rotating machinery. A spectrum-analysis system can help engineers investigate unusual frequency components and changes in machine behavior.



3. Audio and acoustic analysis

Audio samples can be analyzed to examine frequency components, tonal content, and noise. Your hardware architecture could support this type of analysis when the sampling rate, input interface, and processing configuration are appropriate.



4. Embedded sensing and instrumentation

Sensor outputs can be digitized and analyzed to extract useful frequency information. Integrating the processor, DSP hardware, memory access, and communication interfaces provides a foundation for compact embedded measurement systems.

These are potential application areas, not claims that your current prototype has already been tested or validated in all these domains.

Why use a RISC-V CPU with hardware DSP acceleration?

Your project combines two complementary approaches.

RISC-V CPU and software

Configures and coordinates the system

Executes firmware and handles control decisions

Dedicated DSP hardware

Performs the assigned filtering and FFT operations

Processes data through dedicated hardware logic

Reads status and accesses results

Reduces the amount of FFT arithmetic the CPU must execute

Manages communication with the host PC

Works alongside the CPU to accelerate signal processing

A programmable CPU makes the system easier to control and extend. Dedicated DSP IPs can perform their assigned mathematical operations in hardware. The actual performance advantage depends on the implementation, clock frequency, throughput, resource usage, and software overhead; it should be measured rather than assumed.

What is the significance of your project?

The significance is not simply that you have implemented an FFT. It is that you are integrating multiple hardware and software components into a functional embedded processing architecture.

Your work brings together:

- Processor-based control: using a RISC-V CPU to coordinate the system.
- Hardware acceleration: using FIR and FFT IPs for digital signal processing.
- Peripheral and interconnect integration: connecting the IPs through the appropriate bus interfaces.
- Memory and result handling: providing a route for accessing processed data.
- Host communication: using UART to make results accessible outside the chip.

This makes the project relevant to embedded systems design, RTL development, hardware/software integration, and VLSI system architecture.

A complete explanation you can use when presenting your project

Writing

Our project, “**Design and RTL Implementation of a RISC-V Based Spectrum Analyzer SoC with Hardware DSP Acceleration,**” aims to develop an integrated system for analyzing the frequency content of digital signals.

The system receives a real-world analog signal, which is converted into digital samples by an external ADC. These samples enter the ADC Interface and can be processed by a programmable FIR filter to attenuate unwanted frequency components. The FFT accelerator then transforms the time-domain samples into frequency-domain data, which is accessed through the FFT memory interface.

A RISC-V processor coordinates the system by configuring the IPs, issuing control commands, checking processing status, and accessing the results. Supporting peripherals such as UART, GPIO, and Timer provide communication, digital I/O, and timing functions. UART enables selected results to be transmitted to a host computer, where software can display the frequency spectrum.

The problem addressed by this project is the need to analyze the frequency components of sampled signals while integrating signal acquisition, digital filtering, FFT computation, processor control, memory access, and communication in a single SoC architecture. By using dedicated hardware for DSP operations and a programmable processor for system management, the design demonstrates how hardware acceleration can be incorporated into an embedded signal-processing system.

Potential applications include communication-signal analysis, audio processing, vibration monitoring, and sensor-based instrumentation. The project establishes a hardware and software foundation for these applications; application-specific analog interfaces, sampling requirements, firmware, and validation would be needed for a complete practical deployment.

One important distinction for your report: describe the architecture above as the intended end-to-end system, and describe any feature as implemented and verified only when your integrated RTL and test results demonstrate it.